	**Carátula para entrega de prácticas**

Laboratorio de computación salas A y B

Profesor:	M.C Rene Adrian Davila Perez
Asignatura:	Programación orientada a objetos
Grupo:	7
No de Práctica(s):	7
Integrante(s):	323036285 323279866 323209432
	323161891 323063609
No. De Lista o brigada	3
Semestre:	2027-1
Fecha de entrega:	28/ 09 / 2026
Obervaciones:	

CALIFICACIÓN: _____

Índice

1. Introducción **2**

2. Marco Teórico **3**

3. Desarrollo **4**

4. Resultados **11**

5. Conclusión **13**

1. Introducción

El uso de modelos de lenguaje (LLM) como herramienta de consulta académica exige un enfoque crítico. Aceptar sus respuestas automáticamente puede llevar a adoptar malas prácticas de diseño en la programación orientada a objetos. Por ello, es esencial interactuar con la IA para cuestionar decisiones estructurales, como los problemas de acoplamiento al usar modificadores de acceso específicos en la herencia, y analizar la veracidad de la información generada.

Por otra parte, la aplicación de los pilares de la programación es fundamental para evitar código redundante. Al modelar figuras geométricas, una jerarquía profunda y estructurada permite reutilizar atributos y especializar comportamientos. El desarrollo de este programa busca aplicar la herencia y la composición creando un sistema modular.

Objetivos

- Consultar a un LLM por qué no es buena práctica usar encapsulamiento protected en herencia, para verificar la ón que nos dé y lograr progrmas más eficientes.
- Construir una jerarquía de herencia profunda utilizando Cuadrilátero como superclase base, derivando sucesivamente en Trapezoide, Paralelogramo, Rectángulo y Cuadrado, para poner en práctica lo visto en clase y desarrollar nuestras habilidades de programación.
- Aplicar la sobreescritura de métodos mediante el uso de @Override, para asegurar que cada subclase adapte y calcule el área con su fórmula geométrica correspondiente.

2. Marco Teórico

En esta práctica se utilizará la herencia a objetos [1], la cual le permite a las clases adquirir tanto las propiedades como los comportamientos de otras, promoviendo la reutilización de código y el uso de polimorfismo [2].

`extends`: Palabra reservada en Java para declarar una clase que sea heredada de un caso base.

`instanceof()`: Permite afirmar que un objeto es heredado de otra clase.

`toString`: Definido de la clase `Object`, se utiliza para mostrar la información de un objeto.

`@Override`: redefine el comportamiento de un método específico en una subclase, por lo que tendrá un comportamiento acorde a lo que necesita, necesita tener el mismo nombre de la función a la que sobreescribirá.

`Constructores`: Se pueden usar constructores sobrecargados, pero hay que invocarlo explícitamente.

`super`: permite acceder a los elementos de una clase base.

`Sobrecarga`: Puede definir varios métodos con el mismo nombre, pero con diferentes tipos y números de parámetros.

`Sobreescritura`: Se refiere al hecho de redefinir un método heredado.

3. Desarrollo

Ejercicio 1

Para el primer ejercicio se le preguntó a un LLM por qué no es buena práctica usar encapsulamiento protected en herencia, usando el siguiente prompt:

Soy estudiante de ingeniería, explicame en 200 palabras de forma sencilla por que no es buena practica usar encapsulamiento protected en herencia.

A lo que respondió:

Tavera Martinez Javier Jeremias:
El uso de `protected` en la herencia debilita el principio de encapsulamiento en la Programación Orientada a Objetos. Aunque permite que las clases hijas accedan directamente a los atributos de la clase padre, esto genera un acoplamiento fuerte y el conocido problema de la clase base frágil.

Desde una perspectiva de ingeniería de software, la práctica recomendada es mantener los atributos estrictamente privados (`private`). En su lugar, se deben proveer métodos de acceso seguros (como `getters` y `setters`) o aplicar el principio de composición sobre herencia. Esto garantiza un código modular, robusto y fácil de mantener a lo largo del ciclo de vida del software, evitando dependencias ocultas y efectos colaterales indeseados entre los diferentes componentes del sistema.

En los sistemas complejos, aislar los cambios es vital para prevenir fallas en cascada y asegurar que cada clase mantenga sus propias responsabilidades de forma independiente.

Piñón Hernández Leslie Montserrat :
Porque ‘protected‘ rompe el encapsulamiento en la práctica. Es la respuesta corta; los motivos concretos son estos:

- **Acopla la subclase a la implementación.** Si la subclase accede directamente a un atributo ‘protected‘, depende de cómo está hecha la clase padre. Si el padre cambia el tipo, el nombre o la lógica de ese atributo, se rompen todas las hijas.

- **Se pierden las invariantes.** Una clase padre con atributo privado y setter puede validar (por ejemplo, que 'saldo ¿= 0'). Con 'protected', cualquier subclase puede modificarlo directamente y dejar el objeto en un estado inválido.
- **El alcance es mayor del que parece.** En Java, 'protected' también da acceso a todas las clases del mismo paquete, no solo a las hijas. Así que el atributo queda expuesto a más código del esperado.
- ** Cualquiera puede heredar de tu clase en el futuro, y todo lo 'protected' se vuelve parte de tu .APIpública"hacia las subclases. Cambiarlo después es costoso.
- **Va contra el principio de bajo acoplamiento/alta cohesión** y contra la guía de *Effective Java*: "diseña y documenta para la herencia, o prohíbela".

Delgado Flores Jesús Giovani:

"es buena práctica utilizar el encapsulamiento `protected` en herencia al programar en java? explica qué situaciones se generan con esto"

R: Depende de qué marques como `protected`: los métodos con criterio sí, los atributos casi nunca.

- Las subclases pueden romper los invariantes de la clase padre.
- Problema de la clase base frágil (*fragile base class*): Si la subclase usa `saldo` directamente y luego quieres cambiar la representación (por ejemplo, de `double` a `BigDecimal`, o a una lista de movimientos), rompes todas las subclases. Un atributo `protected` es parte del contrato con las subclases, no un detalle interno.
- Fuga de acceso dentro del paquete.
- Se vuelve API pública de facto: Si publicas una librería, cualquiera puede extender tu clase desde otro paquete, así que los miembros `protected` quedan sujetos a compatibilidad hacia atrás igual que los `public`.
- Regla de acceso confusa entre paquetes.

Es recomendable utilizar `private` para los atributos, si la subclase necesita acceso, ofrece métodos `protected` pequeños y con validación (por ejemplo

`protected void depositar(double monto)`) en lugar de exponer el campo. Documenta qué métodos están pensados para sobrescribirse, o declara la clase `final` si no está diseñada para herencia.

Sierra Alcaraz Daniel:
Aunque el modificador protected fue concebido en la programación orientada a objetos para permitir la especialización de clases sin exponer datos al ámbito público, en la práctica suele comprometer la arquitectura del software a mediano y largo plazo. Sustituir un encapsulamiento estricto (private) por accesos protegidos rompe las barreras de contención del estado interno, introduciendo problemas de mantenimiento y concurrencia como los siguientes:

Complica el soporte multihilo (Thread-Safety): Si la superclase implementa sincronización o bloqueos para proteger sus datos, cualquier subclase puede modificar las variables protected de forma directa sin adquirir los bloqueos necesarios, provocando condiciones de carrera (race conditions) imposibles de controlar desde la clase base.

Dificulta la depuración y pruebas unitarias: Cuando el estado interno de un objeto se corrompe en tiempo de ejecución, es difícil rastrear el error porque cualquier clase derivada (o cualquier clase del mismo paquete) pudo haber modificado la variable directamente, eliminando la trazabilidad que ofrece un punto único de depuración (como un breakpoint en un método accesor).

Impide la optimización interna y el cómputo diferido (Lazy Evaluation): Si se expone un campo de forma directa, la superclase pierde la libertad de modificar su lógica de representación interna en versiones futuras (por ejemplo, calcular una propiedad solo bajo demanda o almacenar valores en caché), ya que las subclases asumen que el campo en memoria siempre existe y es accesible tal cual.

Incentiva jerarquías rígidas sobre la composición: Facilita el antipatrón de extender clases solo para acceder a sus datos internos (”herencia por conveniencia”), en lugar de desacoplar responsabilidades mediante composición y delegación, violando directamente el principio de Abierto/Cerrado (Open/Closed Principle).

Toledo Sifuentes Germán:

No es buena práctica principalmente por tres razones:

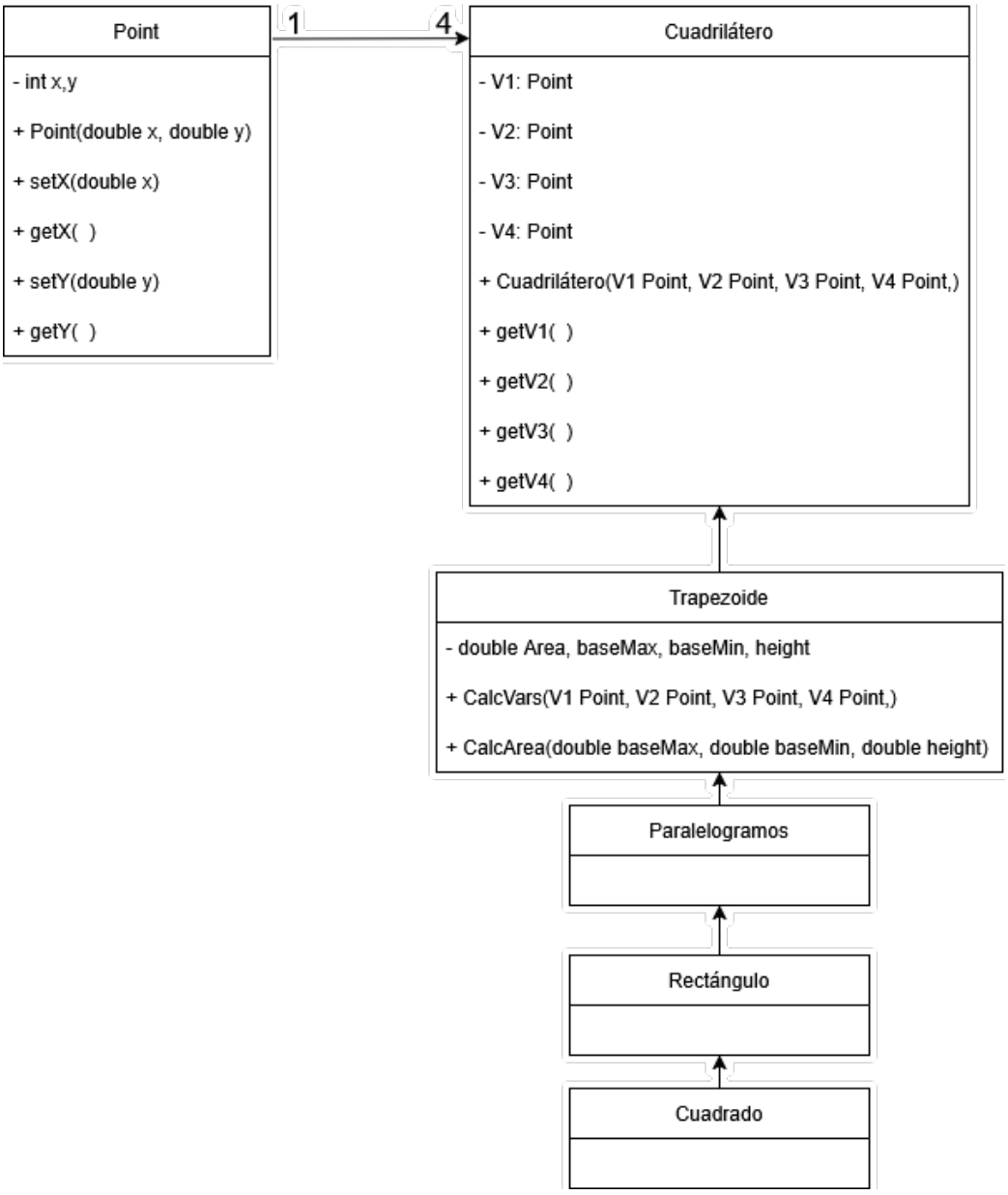
- Rompe el encapsulamiento real: En Java, `protected` no solo da acceso a las subclases, sino también a cualquier otra clase que esté dentro del mismo paquete, dejando los datos expuestos.
- Genera alto acoplamiento: Las subclases dependen directamente de la implementación interna de la clase padre. Si modificas el nombre o tipo de una variable `protected`, romperás todas las clases hijas.
- Pérdida de control e invariantes: Cualquier subclase puede alterar el valor de la variable directamente sin pasar por validaciones, facilitando que el objeto quede en un estado inconsistente o con errores lógicos.

La mejor práctica es declarar los atributos como `private` y exponer su acceso o modificación controlada mediante métodos `getters`, `setters` o constructores (`super(...)`).

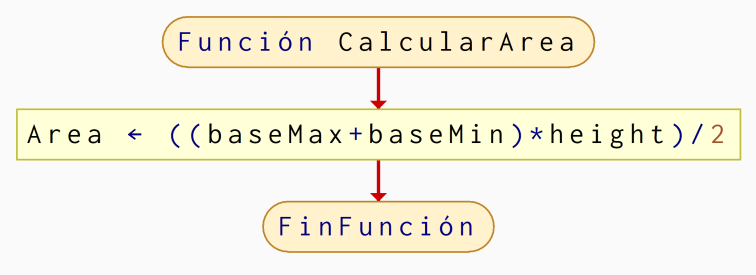
Ejercicio 2

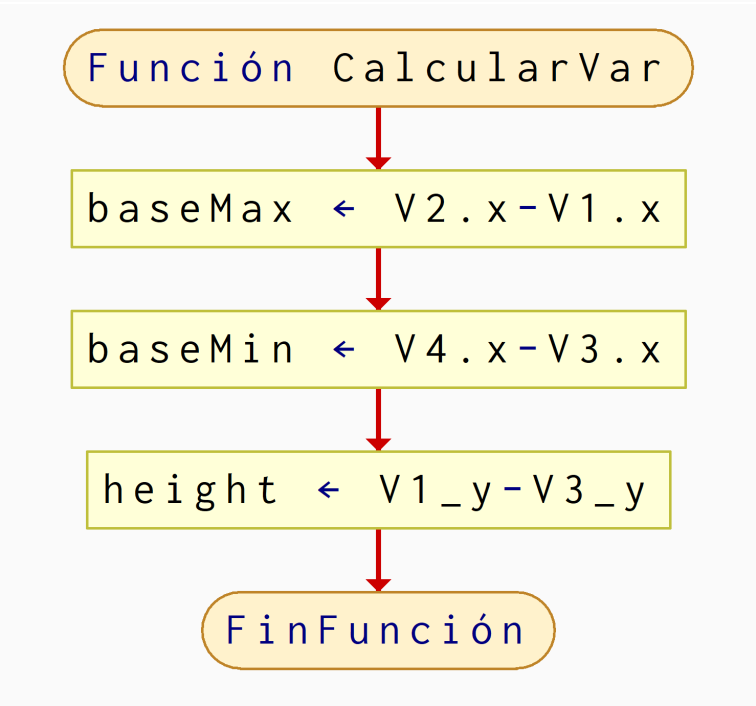
Para el segundo ejercicio se creó un programa que calcula áreas de cuadriláteros empleando herencia:

Se siguió el siguiente UML:



Y los siguientes diagrams::





Para la elaboración del programa, se decidió comenzar por la clase `Punto`, con atributos `x` y `y` de tipo `double`, encapsulados de forma privada. Dentro de la clase se programó su constructor y los getters de cada coordenada.

Posteriormente se pasó a la clase `Cuadrilatero`, la cuál tiene como atributos cuatro objetos de tipo `Punto`, encapsulados de forma privada. En ésta clase se programó un constructor, donde se establecen los cuatro vértices de la figura y se crearon getters para cada punto. Finalmente se realizó un método para imprimir los vértices usando getters de la clase `Punto` para obtener las coordenadas.

Después de la clase `Cuadrilatero` se usó `extends` para crear una clase hija `Trapezoide`, la cuál hereda sus métodos y atributos. Se programó un constructor, donde se utilizó `super` para hacer que la construcción se realice con el método de la clase `Cuadrilatero`. Finalmente se programó un método donde se usa `Math.abs` para obtener un valor absoluto para cada distancia y getters para obtener cada punto y con todo ésto finalmente se calcula su área con `return ((baseMayor + baseMenor) * altura) / 2.0`.

Posteriormente se creó una clase hija de `Trapezoide` llamada `Paralelogramo`. Se programó un constructor, donde se utilizó `super` para hacer que la construcción se realice con el de la clase `Trapezoide`. Finalmente se programó un método, el cuál sobreescribe lo que había en la clase `Trapezoide` usando `@Override`, donde se usa `Math.abs` para obtener un valor absoluto para cada distancia y getters para obtener cada punto y con todo ésto finalmente se calcula su área con `return base * altura`.

Tras la programación de `Paralelogramo`, se procedió con una clase hija, llamada `Rectángulo`. Se realizó un constructor, donde se utilizó `super` para que la construcción se realice con el de la clase `Paralelogramo`. Se programó un método `CalcularArea`, donde se usa `Math.abs` para obtener un valor absoluto para cada distancia y getters para obtener cada punto y con todo ésto finalmente se calcula su área con `return base * altura`. Finalmente se usó `@Override` para sobreescribir lo que está por encima y calcular el área del rectángulo.

Para finalizar con los métodos que invocará `Main`, se hizo la clase `Cuadrado`. Se realizó un constructor, donde se utilizó `super` para que la construcción se realice con el de la clase `Rectángulo`. Finalmente se programó un método para calcular su área, sobreescribiendo lo que está por encima mediante el uso de `@Override`, donde se usa `Math.abs` y `get` para calcular un solo lado y obtener su área mediante `return lado * lado`.

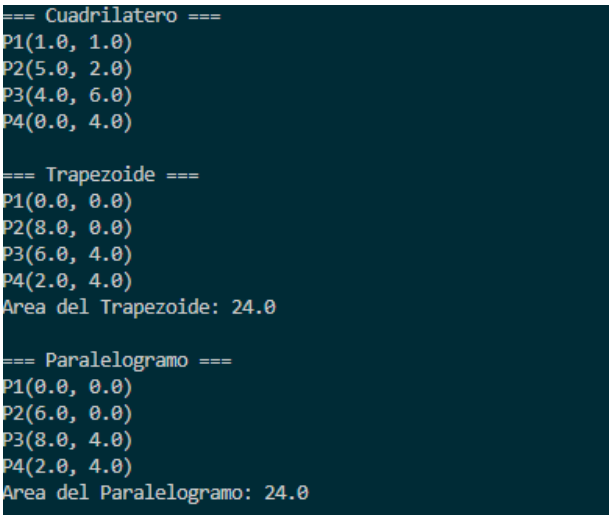
Para finalizar el programa, en el método `Main`, se fueron asignando puntos e invocando métodos de de diferentes clases, para finalmente imprimir los vértices y el área de cada figura.

4. Resultados

Para comprobar el funcionamiento correctos, en la consola podemos observar la ejecución de las diferentes figuras geométricas implementadas:

Cuadrilátero: Se imprimen de forma correcta las coordenadas de sus cuatro vértices definidos en la clase.

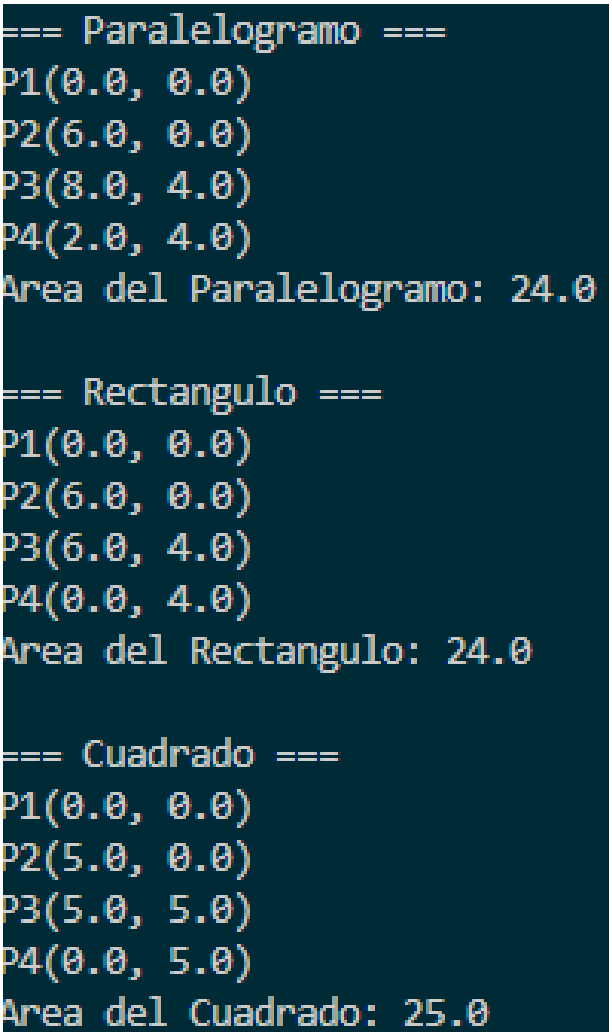
Trapezoide: Muestra las coordenadas de sus cuatro vértices y calcula de manera exacta el área total, obteniendo un valor de `24.0`.



Paralelogramo: Vemos impresos sus cuatro vértices y el cálculo exitoso de su área con un valor de `24.0`.

Rectángulo: Muestra las coordenadas correspondientes a sus vértices rectos y el cálculo correcto de su área, obteniendo también `24.0`.

Cuadrado: Despliega los cuatro puntos simétricos que lo conforman y calcula de manera exacta su área, dando como resultado `24.0`.



Con esto corroboramos que la implementación de las clases, la herencia, el manejo de puntos en el plano cartesiano y las fórmulas de área funcionan de manera correcta.

5. Conclusión

Es importante conocer los conceptos teóricos ya que al modelar los diferentes tipos de cuadriláteros como una cadena de clases en la que cada una es un caso específico a la anterior nos permite con la herencia reutilizar atributos y métodos sin duplicar código . La utilización de cada uno de los conceptos como super, Override y el encapsulamiento nos permiten facilitar cada clase que se herede del padre y con los resultados obtenidos confirman el planteamiento de las figuras con los vértices y el área correctos , lo que demuestra que combinar estos mecanismos da como resultado un código más ordenado, reutilizable y fácil de extender.

Referencias

[1] GeeksforGeeks. *Inheritance in Java*. https://www.geeksforgeeks.org/java/inheritance-in-java/. [En línea].

[2] J. A. Solano. *Herencia*. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3057/mod_resource/content/1/UAPA-Herencia/index.html. Unidades de Apoyo para el Aprendizaje, CUAED/Facultad de Ingeniería-UNAM, 2020.